CARTER

PRODUCT REQUIREMENTS DOCUMENT

Fully Offline Personal AI Agent,
Local ML Lab, and Model Training Workbench

Version	1.0
Status	Proposed — implementation baseline
Prepared	26 October 2026
Target platform	ASUS TUF A15 • Windows 11
Hardware target	RTX 3050 4 GB VRAM • 16 GB RAM
Delivery plan	8 weeks; MVP targeted by Week 6

Core principle: local-first, bounded autonomy, measurable behavior.

Product Requirements Document (PRD)

Fully Offline Personal AI Agent, Local ML Lab, and Model Training Workbench

Document version: 1.0

Status: Proposed — implementation baseline

Prepared: 26 October 2026

Target platform: ASUS TUF A15, Windows 11, NVIDIA RTX 3050 4 GB VRAM, 16 GB RAM

Project duration: 8 weeks (MVP in 6 weeks; hardening and stretch goals in weeks 7–8)

---

1. Executive Summary

Carter is a privacy-first, fully offline AI system that runs on a personal computer without relying on cloud inference APIs or internet access at runtime. Carter combines a locally hosted language model, a custom machine-learning pipeline, a constrained tool-using agent, persistent local memory, and an experimentation workflow for fine-tuning and training small models from scratch.

Carter is designed for a resource-constrained laptop. The first release prioritizes a dependable local assistant and a safe, auditable agent loop over unrestricted autonomy or large-model training. A compact quantized LLM handles language understanding and planning; allowlisted Python tools perform bounded actions; a separately trained ML model provides a demonstrable predictive capability; and evaluation tests measure reliability and offline readiness.

The machine is intended to be **permanently air-gapped after a verified preparation phase**. All installers, source packages, model files, tokenizers, datasets, licenses, and documentation must therefore be acquired and validated before the final disconnect. Carter must fail clearly when a required local resource is absent rather than attempting to download it.

2. Product Vision

Deliver a local AI workspace that can understand a user's request, propose a plan, call approved local tools, inspect results, save useful notes, and report what it did—while keeping models, data, logs, and inference on the device.

2.1 Product principles

- **Offline by design:** runtime functionality must not require internet connectivity.
- **Local-first privacy:** user prompts, documents, model weights, training data, and logs stay on the device.
- **Bounded autonomy:** Carter may use only explicit, validated tools; it must not execute arbitrary model-generated shell commands or code.
- **Measurable behavior:** model and agent changes must be evaluated against fixed test sets.
- **Reproducibility:** keep model versions, dataset hashes, configurations, random seeds, and evaluation reports.
- **Resource awareness:** prioritize small models, quantization, short contexts, efficient training, and clear memory limits.
- **Human control:** consequential actions require confirmation; stop conditions and action limits are mandatory.

3. Problem Statement

Cloud AI tools can be unavailable, unsuitable for sensitive local data, or incompatible with an air-gapped environment. A local model alone does not provide a complete agent: it needs controlled tools, memory, model evaluation, data handling, and reliable installation procedures. On a laptop with 4 GB VRAM and 16 GB system RAM, model selection and training strategy must also be realistic.

Carter addresses these issues by packaging inference, bounded tool execution, local memory, ML training, and validation into a single reproducible offline project.

4. Goals and Non-Goals

4.1 Goals

- Run an instruction-tuned LLM from local model files.
- Plan and complete bounded multi-step tasks using approved local tools.
- Read and list files inside a designated workspace.
- Save and retrieve notes through local SQLite storage.

- Train a small ML model from scratch on a local dataset, save it, reload it, and evaluate it.
- Provide an optional local inference endpoint bound to loopback (`127.0.0.1`).
- Fine-tune a compatible small LLM with LoRA if the validated hardware/software configuration supports it.
- Include an educational miniature Transformer training experiment from random initialization as a stretch goal.
- Demonstrate operation with all external network interfaces disconnected.
- Provide clear setup, recovery, evaluation, and troubleshooting instructions.

4.2 Non-goals for v1

- Training a competitive general-purpose LLM from scratch.
- Guaranteed fine-tuning of any 3B model on 4 GB VRAM.
- Unrestricted shell, PowerShell, browser, administrator, or filesystem access.
- Internet search, cloud APIs, remote telemetry, or cloud storage.
- Autonomous destructive actions, purchases, account changes, or execution of untrusted code.
- Production-grade multi-user deployment.
- Claims that generated responses are always correct.

5. Target User and Primary Use Cases

Primary user: an individual developer/student who wants to learn, experiment, and perform local tasks with a private AI assistant.

Core use cases

- Ask Carter questions using a local LLM.
- Ask Carter to list or summarize permitted workspace files.
- Ask Carter to save a note and retrieve it later.
- Ask Carter to run a local ML prediction on a supplied sample.
- Train and evaluate a custom ML model without internet access.
- Fine-tune a compatible local language model and compare it with the base model.
- Inspect logs and reproduce a prior experiment.
- Verify that Carter remains functional after reboot with network access disabled.

6. Target Hardware and Technical Constraints

6.1 Known target

- Laptop: ASUS TUF A15
- Operating system: Windows 11
- GPU: NVIDIA RTX 3050 with 4 GB VRAM
- System RAM: 16 GB
- Storage: use the currently available SSD space; confirm before allocating datasets and checkpoints
- Runtime condition: permanently air-gapped after preparation and acceptance testing

6.2 Resource strategy

- Start with a 1B–1.5B instruct model for reliable local inference; evaluate a 3B model only if memory tests pass.
- Prefer 4-bit GGUF for inference with a validated `llama.cpp` build.
- Keep the inference context initially around 2,048 tokens; increase only after memory testing.
- Use CPU inference as a fallback when GPU offload exceeds VRAM.
- Start custom ML with scikit-learn or a small PyTorch network.
- Treat LoRA fine-tuning as conditional on a successful offline compatibility test.

- Use a small sequence length and micro-batch size for fine-tuning; do not assume QLoRA will work on Windows or on this GPU without a validated quantization backend.
- Train a miniature language model from scratch only as an educational stretch goal.

6.3 Storage budget (planning estimate)

Asset	Budget
Installers and build tools	2–5 GB
Python wheelhouse and dependencies	5–15 GB
One or more local LLMs and tokenizers	3–8 GB
Datasets and prepared samples	2–10 GB
Checkpoints, adapters, logs, and reports	5–15 GB
Runtime, source, documentation, and safety copies	2–5 GB
Recommended planning allocation	**30–60 GB**

These figures are not guarantees; actual sizes depend on the selected model, wheel compatibility, datasets, and retained checkpoints. Maintain a separate verified copy of the offline kit on external storage where possible.

7. Product Scope and Functional Requirements

Priority labels: **P0 = required for MVP**, **P1 = important after MVP**, **P2 = stretch goal**.

7.1 Local model runtime

- **FR-01 (P0):** Load an LLM exclusively from a configured local path.
- **FR-02 (P0):** Expose inference to the agent through a local interface bound to `127.0.0.1`.
- **FR-03 (P0):** Provide configurable context length and generation limits.
- **FR-04 (P0):** Fail with a readable error when model files are missing or invalid.
- **FR-05 (P0):** Do not download models or call remote inference services at runtime.
- **FR-06 (P1):** Support switching between validated local model profiles.
- **FR-07 (P1):** Record model identifier, quantization, runtime version, and inference settings for each evaluation run.

7.2 Agent and planner

- **FR-08 (P0):** Convert a user request into a bounded action or sequence of actions.
- **FR-09 (P0):** Use only registered tools with strict argument schemas.
- **FR-10 (P0):** Enforce a configurable maximum of five agent steps in the MVP.
- **FR-11 (P0):** Reject malformed JSON, unknown tools, unexpected arguments, invalid paths, and oversized inputs.
- **FR-12 (P0):** Treat retrieved file contents and tool outputs as untrusted data, not as privileged instructions.
- **FR-13 (P0):** Show a concise final answer and indicate tool failures rather than silently claiming success.
- **FR-14 (P1):** Display a plan and request approval for actions designated as consequential.
- **FR-15 (P1):** Record a local audit trail of actions, arguments (redacted where needed), results, and errors.
- **FR-16 (P1):** Support cancellation and a hard action/time budget.

7.3 Tool registry

MVP tools:

- `list_files`: list files within the configured workspace.
- `read_file`: read small UTF-8 text files within the workspace.
- `save_note`: store a user-approved note in SQLite.
- `search_notes`: search previously saved notes.
- `predict_anomaly`: invoke the locally saved ML model on validated feature input.

- `get_system_status`: report configured local model and ML-model readiness without exposing secrets.

Requirements:

- **FR-17 (P0):** Tools are explicitly allowlisted.
- **FR-18 (P0):** Canonicalize paths and reject path traversal and symbolic-link escapes.
- **FR-19 (P0):** Limit file size, output size, execution time, and request size.
- **FR-20 (P0):** Do not expose arbitrary shell execution, unrestricted Python evaluation, or administrator actions.
- **FR-21 (P1):** Require explicit confirmation for writes outside the notes database or for any future destructive action.

7.4 Persistent memory

- **FR-22 (P0):** Store notes and basic task history in SQLite.
- **FR-23 (P0):** Persist data across process restarts and reboots.
- **FR-24 (P0):** Provide a way to inspect and delete stored notes.
- **FR-25 (P1):** Support optional local document indexing; do not add vector databases until a demonstrated need exists.
- **FR-26 (P1):** Keep secrets out of prompts, logs, and training datasets.

7.5 ML training and evaluation

- **FR-27 (P0):** Provide a deterministic training script for a small ML model.
- **FR-28 (P0):** Split data into train, validation, and test sets without leakage.
- **FR-29 (P0):** Save the trained model, configuration, dataset version/hash, seed, and metrics.
- **FR-30 (P0):** Provide a separate inference script that loads the saved model.
- **FR-31 (P0):** Report precision, recall, F1, and confusion matrix for a classification task; choose appropriate metrics for other tasks.
- **FR-32 (P1):** Support a small PyTorch model trained from random initialization.
- **FR-33 (P1):** Compare a baseline model with a more complex model using the same held-out data.

7.6 LLM fine-tuning

- **FR-34 (P1):** Provide a reproducible LoRA fine-tuning workflow for one locally stored, compatible base model.
- **FR-35 (P1):** Store adapters separately from base weights.
- **FR-36 (P1):** Evaluate base and fine-tuned versions on identical held-out prompts.
- **FR-37 (P1):** Record peak memory, training duration, loss, configuration, and known limitations.
- **FR-38 (P1):** Skip or defer GPU fine-tuning if a tested dependency stack or adequate memory is unavailable.
- **FR-39 (P2):** Attempt QLoRA only with a validated backend and known compatible model architecture.

7.7 Training a language model from scratch

- **FR-40 (P2):** Include a small educational decoder-only Transformer experiment.
- **FR-41 (P2):** Train a tokenizer or document a fixed tokenizer choice.
- **FR-42 (P2):** Plot or record training and validation loss.
- **FR-43 (P2):** Save checkpoints and generate sample text after training.
- **FR-44 (P2):** Clearly label the model as an educational small model, not a general-purpose assistant.

8. User Workflows

Workflow A — First boot and readiness check

- User launches Carter's local launcher.
- Carter checks required model paths, runtime availability, database access, and ML artifact availability.
- Carter verifies that all configured paths exist and that local endpoints are reachable.
- Carter shows a readiness summary: LLM ready/not ready, ML model ready/not ready, memory ready/not ready.
- If a component is unavailable, Carter identifies the missing local resource and does not attempt a download.

Success condition: readiness checks complete without internet access and report component status accurately.

Workflow B — Complete a simple task

- User enters a request.
- Carter identifies whether it can be handled directly or requires a tool.
- The planner emits one structured action.
- The action schema and permissions are validated.
- The tool runs with time, path, and output limits.
- The result is returned to the LLM as untrusted data.
- Carter either requests the next allowed step or returns a final answer.
- The action limit is enforced; errors are recorded locally.

Success condition: the task is completed or stopped safely, and the final response does not claim an action succeeded when it failed.

Workflow C — Train and use a custom ML model

- User places a dataset in the designated data folder.
- A validation script checks file format, required columns, missing values, and label distribution.
- The training script creates reproducible train/validation/test splits.
- The model trains and produces metrics.
- Artifacts and metadata are saved.
- The evaluation script checks performance on held-out data.
- The inference tool loads the saved artifact and validates new inputs.
- Carter can call the prediction tool through its allowlisted interface.

Success condition: model reload and inference work in a fresh process without internet access.

Workflow D — Fine-tune the local LLM

- Select a locally stored, licensed base model.
- Verify tokenizer, model architecture, dependencies, and available memory.
- Validate and format the instruction dataset.
- Separate training and evaluation examples.
- Run a short smoke test before full training.
- Train LoRA adapters and monitor loss/memory.
- Save adapter and tokenizer artifacts.
- Evaluate base and fine-tuned models on identical held-out prompts.
- Retain the adapter only if the measured result justifies it.

Success condition: the adapter can be loaded with the base model offline and produces reproducible evaluation results. Fine-tuning is not considered a release blocker if the target hardware cannot support the validated configuration.

Workflow E — Air-gap acceptance test

- Freeze and checksum the offline kit.
- Disconnect all external network paths.
- Restart the laptop.
- Install from local installers and wheelhouse in a clean environment.
- Run model inference, agent tasks, ML training, ML inference, and memory persistence tests.
- Inspect logs and network behavior for unexpected external access attempts.
- Reboot and repeat smoke tests.
- Sign off only after all P0 acceptance criteria pass.

Success condition: Carter completes the required scenarios with external connectivity unavailable and does not depend on a remote resource.

9. Safety, Privacy, and Security Requirements

- Default to least privilege and a dedicated workspace directory.
- Bind local inference services to loopback only; do not expose them to the LAN.
- Disable remote telemetry and automatic updates where possible.
- Do not allow the LLM to run arbitrary shell commands or generated scripts.
- Validate every tool name, argument, path, data type, and size before execution.
- Treat documents as potentially hostile prompt-injection sources.
- Require confirmation for future actions that delete or overwrite user data.
- Keep API keys and passwords out of prompts, logs, and datasets; ideally, no cloud credentials should be present on the air-gapped machine.
- Keep a read-only or separately stored backup of the installation kit, base models, datasets, and known-good artifacts.
- Use software and model licenses that permit the intended use.
- Verify checksums and scan files before transferring them to the isolated machine.
- Document that offline operation reduces network exposure but does not eliminate risks from malicious files, compromised installers, removable media, or model errors.

10. Non-Functional Requirements

ID	Requirement	Target
NFR-01	Offline operation	Core MVP works with external network disconnected
NFR-02	Local privacy	No cloud inference or remote storage
NFR-03	Startup	Readiness check completes within 30 seconds on the target laptop, excluding model loading where slower
NFR-04	Reliability	Invalid model responses stop safely rather than invoking an unknown tool
NFR-05	Agent bounds	Maximum 5 tool steps in MVP, configurable
NFR-06	Resource control	Model/context settings documented and tested against VRAM/RAM
NFR-07	Reproducibility	Record versions, seed, data hash, configuration, and metrics
NFR-08	Recovery	Known-good model and environment can be restored from local backup
NFR-09	Observability	Local logs capture errors and action outcomes without unnecessary sensitive content
NFR-10	Maintainability	Separate agent, tools, ML training, LLM runtime, and evaluation modules

Latency depends on model, quantization, CPU/GPU offload, context length, and thermal conditions; no hard tokens-per-second guarantee is made before benchmarking.

11. Architecture and Technology Stack

Layer	Proposed technology	Role
Operating system	Windows 11	Host environment
Language	Python 3.12	Agent, ML, evaluation and scripts
Local inference	`llama.cpp` with GGUF	Quantized local LLM serving
ML baseline	scikit-learn, NumPy, Pandas, joblib	Custom model training and inference
Deep learning	PyTorch	Neural network training
Fine-tuning	Transformers, PEFT, Accelerate; optional validated quantization backend	LoRA/QLoRA experiments
Agent orchestration	Small custom Python loop with structured tool schemas	Planner and tool routing
Memory	SQLite + JSON	Notes and local audit records
Testing	pytest and standard-library tests	Regression and offline checks
Interface	Terminal for MVP; optional local web UI later	User interaction
Versioning	Git local repository	Code and experiment history

Keep runtime dependencies minimal. Install only what is needed for each milestone; separate inference dependencies from training dependencies if that reduces conflicts and storage.

12. Eight-Week Delivery Timeline

The schedule assumes focused work alongside coursework. Each week has a tangible deliverable and exit criteria.

Week 1 — Requirements and offline supply chain

Tasks

- Confirm available SSD space and create a dedicated project directory.
- Select one primary small instruct model and one fallback model.
- Confirm model licenses, quantization, tokenizer files, and runtime compatibility.
- Prepare pinned Python dependencies and wheelhouse on a connected staging computer.
- Create checksums, inventory, setup notes, and a clean-environment installation test.

Deliverable: verified offline installation kit and architecture baseline.

Exit criteria: all required files are present; a clean installation test passes before the final air-gap.

Week 2 — Local inference foundation

Tasks

- Install Python and the validated runtime from local media.
- Load the primary GGUF model.
- Benchmark CPU and GPU offload, RAM/VRAM use, context length, and latency.
- Add a model readiness check and clear failure messages.
- Test after reboot with networking disconnected.

Deliverable: reliable local LLM inference.

Exit criteria: a fixed prompt set runs offline and all model files are loaded locally.

Week 3 — Agent loop and safety controls

Tasks

- Implement structured action format and schema validation.
- Add the tool registry, step budget, error handling, and cancellation.
- Implement `list_files`, `read_file`, and `save_note`.
- Add workspace boundary checks, output limits, and local action logs.
- Test malformed responses and path traversal attempts.

Deliverable: safe bounded tool-using agent.

Exit criteria: approved tasks work; unknown actions and invalid paths are rejected.

Week 4 — Custom ML model and integration

Tasks

- Select or prepare a small local dataset.
- Implement dataset validation and leakage-resistant splits.
- Train a baseline model from scratch.
- Save model, metadata, and evaluation results.
- Implement `predict_anomaly` and integrate it with Carter.
- Test model reload in a fresh process.

Deliverable: end-to-end local ML training and prediction.

Exit criteria: held-out metrics are reported and offline prediction works after restart.

Week 5 — Persistent memory and regression testing

Tasks

- Add SQLite notes and task history.
- Add search/retrieval for notes and a way to inspect/delete them.
- Create fixed agent evaluation scenarios.
- Add regression tests for all tools and failure cases.
- Test local service startup and error recovery.

Deliverable: persistent assistant with repeatable tests.

Exit criteria: memory survives reboot and regression tests pass.

Week 6 — Fine-tuning feasibility and MVP release

Tasks

- Validate the base model, tokenizer, dataset format, and training stack offline.
- Run a short LoRA smoke test.
- If feasible, train a small adapter and compare with the base model.
- If GPU memory is insufficient, document the limitation and use a smaller model or defer fine-tuning.
- Freeze a stable MVP and write user documentation.

Deliverable: Carter MVP release candidate.

Exit criteria: inference, agent, memory, ML training/evaluation, and air-gap tests pass. Fine-tuning has either passed a feasibility test or is explicitly deferred with evidence.

Week 7 — Fine-tuning improvement and scratch Transformer

Tasks

- Improve instruction data quality and held-out evaluation.
- Run a more complete LoRA experiment if feasible.
- Implement a miniature decoder-only Transformer training experiment.
- Record training and validation loss and save checkpoints.

Deliverable: documented fine-tuning comparison and educational scratch-model experiment.

Exit criteria: experiments are reproducible and clearly separated from the stable MVP.

Week 8 — Hardening, documentation, and final acceptance

Tasks

- Run the full air-gap acceptance suite after reboot.
- Verify checksums and restore from the backup kit.
- Review local logs for unexpected network attempts and sensitive data.
- Test malformed tool requests, hostile file content, missing files, and resource exhaustion.
- Document limitations, recovery steps, model licenses, and future improvements.

Deliverable: Carter v1.0, offline deployment kit, test report, and user guide.

Exit criteria: all P0 acceptance tests pass, known limitations are documented, and recovery is demonstrated.

13. MVP Acceptance Criteria

Carter v1.0 is accepted when all of the following are true:

- ■ The LLM loads from a local file without downloading anything.
- ■ The local inference endpoint binds to `127.0.0.1`.
- ■ Carter can complete at least 4 of 5 predefined safe agent scenarios in repeated evaluation runs; failures are recorded.
- ■ Unknown tools, malformed arguments, invalid paths, and excessive requests are rejected.
- ■ The agent respects its five-step limit.
- ■ Notes persist across process restart and laptop reboot.
- ■ A custom ML model trains locally and saves its artifact.
- ■ The saved ML model reloads in a fresh process and produces a prediction.
- ■ Evaluation reports include the relevant held-out metrics and configuration.
- ■ The full P0 suite passes with external networking disconnected.
- ■ Missing dependencies or model files produce a clear error rather than an attempted download.
- ■ The offline kit and known-good artifacts have verified backups.

Fine-tuning and scratch-Transformer training are stretch milestones rather than mandatory conditions for the core MVP, given the hardware constraints.

14. Risks and Mitigations

Risk	Impact	Mitigation
Missing package or incompatible wheel	Offline installation fails	Pin versions, build a complete wheelhouse, test in a clean environment
GPU out-of-memory	Inference or training fails	Smaller model, 4-bit inference, shorter context, CPU fallback
QLoRA backend incompatibility	Fine-tuning blocked	Validate exact OS/backend before air-gap; make fine-tuning conditional
LLM emits invalid actions	Agent reliability or safety issue	Constrained output, schema validation, allowlist, step budget
Prompt injection in local documents	Agent may attempt unsafe actions	Treat content as untrusted, limit tools, require confirmation
Dataset leakage or poor labels	Misleading metrics	Fixed splits, data review, held-out evaluation
Disk exhaustion	Checkpoints or logs fail	Storage budget, cleanup policy, disk-space checks
Corrupted removable media	Kit unavailable	Checksums and a second verified backup
Model license restrictions	Use may be non-compliant	Record and review each model's license before transfer
False confidence from synthetic data	Poor real-world performance	Label synthetic experiments as demos; evaluate on representative data

15. Definition of Done

The project is done when:

- Carter starts and reports readiness without internet.
- The local LLM answers prompts and does not depend on cloud inference.
- The bounded agent performs approved tasks, logs outcomes, and rejects invalid actions.
- Persistent memory works across restarts.
- The custom ML model can be trained, evaluated, saved, reloaded, and called by Carter.
- Fine-tuning has a tested path or a documented hardware-based deferral.
- The scratch Transformer experiment is delivered if time and resources permit.
- A clean offline installation and reboot test pass.
- The project includes source code, pinned dependency inventory, model/data manifest, licenses, tests, recovery guide, and final evaluation report.

16. Final Product Positioning

Carter is a local AI agent and learning lab—not a cloud chatbot clone. Its value is the integration of local inference, controlled actions, custom ML training, reproducible evaluation, and genuine offline operation. The implementation should favor a small reliable system with measurable behavior over a larger model with unverified capabilities.

First action

Before coding, complete the Week 1 supply-chain checklist. Select the exact base model and runtime, verify every required file, and perform a clean installation rehearsal. The offline constraint makes preparation and verification part of the product—not an administrative afterthought.